

Match the tiles

Practical Work/Assignment No. 1
T1c-g1

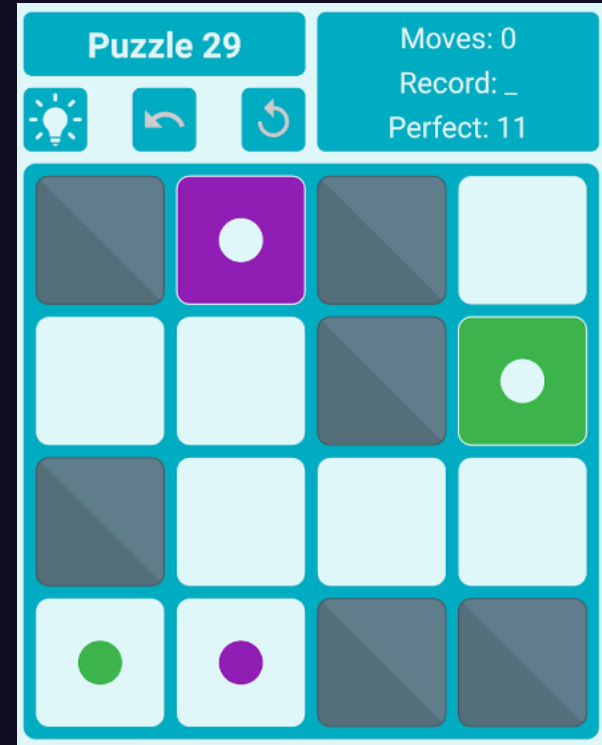
Descrição do jogo

O jogo consiste em colocar todas as peças coloridas nos locais designados com a mesma cor.

De modo a resolver o quebra-cabeças, os movimentos são sincronizados e os espaços entre as peças são criados pela peças fixas existentes.

Existem vários níveis e portanto diferentes configurações do jogo.

Podem existir também limitações tais como tempo limite ou número limitado de jogadas.



► Pesquisa de material relacionado

01

<https://www.appbrain.com/app/match-the-tiles-sliding-game/net.bohush.match.tiles.color.puzzle>

02

<https://blog.goodaudience.com/solving-8-puzzle-using-a-algorithm-7b509c331288>

03

<https://filebox.ece.vt.edu/~ECE2574/programs/P5/index.html>

04

<https://visualstudiomagazine.com/articles/2015/10/30/sliding-tiles-c-sharp-ai.aspx>

05

<https://www.almabetter.com/bytes/tutorials/artificial-intelligence/8-puzzle-problem-in-a>

06

<https://ics.uci.edu/~kkask/Fall-2016%20CS271/slides/03-InformedHeuristicSearch.pdf>

► Formulação do problema como search problem

State Representation:

- Board - matriz;
- Peças - (x(linha),y(coluna));
 - 0-vazio;
 - 1-peças de jogo;
 - 2-peças fixas;
 - 3-final;

Initial State:

- Matriz A preenchida com o estado inicial desejado;

Objective State:

- Colocar todas as peças coloridas nos locais designados com a mesma cor. (função match)

	Pré condições	Efeitos	custo
cima	While (board[self.x+1][self.y+1-1]!=3 and board[self.x+1][self.y+1-1]!=1 and self.y>0)	board[self.x+1][self.y+1-1]=1 board[self.x+1][self.y+1]=0 self.y-=1	Interações do ciclo while
baixo	while (board[self.x+1][self.y+1+1]!=3 and board[self.x+1][self.y+1+1]!=1 and self.y<=n)	board[self.x+1][self.y+1+1]=1 board[self.x+1][self.y+1]=0 self.y+=1	Interações do ciclo while
direita	while (board[self.x+1+1][self.y+1]!=3 and board[self.x+1+1][self.y+1]!=1 and self.x<=n)	board[self.x+1+1][self.y+1]=1 board[self.x+1][self.y+1]=0 self.x+=1	Interações do ciclo while
esquerda	while (board[self.x+1-1][self.y+1]!=3 and board[self.x+1-1][self.y+1]!=1 and self.x>0)	board[self.x+1-1][self.y+1]=1 board[self.x+1][self.y+1]=0 self.x-=1	Interações do ciclo while

► Jogo

Development environment:

- Linguagem de programação: Python
- Bibliotecas: Pygame (para a interface gráfica)
- IDE: atom

Data structures:

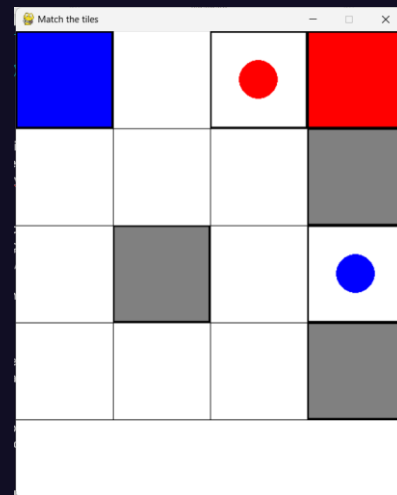
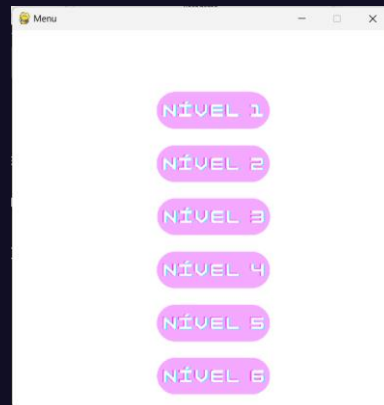
- Classe peça para representar cada peça do quebra-cabeça, contendo informações como posição atual, movimento, etc.
- Matriz Board para representar o estado atual do tabuleiro, contendo informações sobre a configuração atual das peças.

Algorithms:

- Verificar se o jogo terminou (se as peças estão na posição correta)(função match).
- Movimentar as peças do quebra-cabeça.(função move)

Others:

- Opção para permitir que o jogador reinicie o jogo a qualquer momento.



► Jogo (jupyter)

Development environment:

- Linguagem de programação: Python
- Bibliotecas: Pygame (para a interface gráfica)
- IDE: vscode

Data structures:

- Classe Match para representar cada peça que se move, contendo informações como posição atual, movimento, etc.

Algorithms:

- BFS
- A*
- Greedy

Heuristic:

- Manhattan distances
- Linha correta (peça está na mesma linha que a posição final)

► Heurísticas

```
def h1(state):      #retorna 1 caso a peca esteja na mesma linha que a posição final
    board=state.board
    side = len(board)
    for row in range(side):
        for col in range(side):
            if board[row][col]==1 or board[row][col]==2:
                for n in board[row]:
                    if n==4 or n==5:
                        return 1
    return 0
```

```
def h2(state):
    # heuristic function
    # retorna a distancia de manhattan de cada peca à posicao final
    board = state.board
    side = len(board) # the size of the side of the board (only for square boards)
    a,b,c,d =find_peca_fixas()

    p1=abs(state.peca1_row - a) + abs(state.peca1_col - b)
    p2=abs(state.peca2_row - c) + abs(state.peca2_col - d)
    total=p1+p2

    return total
```

► Resultados experimentais

prob1

breadth-first:	1 rounds	0.001s
greedy h2:	1 rounds	0.0s
greedy h1:	1 rounds	0.001s
a-star h1:	1 rounds	0.0s
a-star h2:	1 rounds	0.0014s

prob2

breadth-first:	5 rounds	0.2657s
greedy h2:	7 rounds	0.0042s
greedy h1:	7 rounds	0.0047s
a-star h1:	5 rounds	0.0097s
a-star h2:	6 rounds	0.0055s

► Conclusões

- Com base na análise dos resultados experimentais conseguimos observar que, apesar de no prob1 ter um desempenho satisfatório, o breath-first-search seria a opção menos eficiente.
- No prob1 destacam-se os algoritmos greedy com heurística 2 e o A* com a heurística 1 (ambos com uma execução de 0.0s).
- No prob2 o melhor seria o greedy com a heurística 2, embora alternativas como greedy com a heurística 1 e A* com heurística 2 também sejam opções com resultados próximos.
- Avaliando ambos os problemas com os diversos algoritmos e heurísticas, se fosse necessário selecionar apenas um, optaríamos pelo greedy com a heurística 2 (soma de Manhattan). Essa escolha é baseada na sua eficácia nos problemas analisados.

	Prob1	Prob2
BFS	0,001s	0,2657s
Greddy h2	0,0s	0,0042s
Greddy h1	0,001s	0,0047s
A* h1	0,0s	0.0097s
A* h2	0,0014s	0,0055s

► Greedy com soma de Manhattan

- A heurística da distância de Manhattan é computacionalmente eficiente para calcular e fornecer uma boa estimativa do custo para alcançar o estado final. Essa eficiência é particularmente benéfica neste algoritmo que prioriza ganhos imediatos sem considerar extensivamente as consequências futuras.
- A distância de Manhattan é admissível para problemas com estruturas bidimensionais, ou seja, nunca superestima o custo para alcançar o objetivo. Essa propriedade garante sempre que se mova em direção ao estado final, embora nem sempre encontre a solução ótima.
- Algoritmos Greedy normalmente têm menor complexidade espacial em comparação a algoritmos como A^* ou pesquisa em largura (BFS).

► Referências

- https://youtu.be/GMBqjxcKogA?si=_vZRb74f7ii1h3Nb
- https://youtu.be/2iyx8_elcYg?si=H8l2GuAt_FBTMsgF
- <https://youtu.be/yEoorP-9yQs?si=1d3d2BdbNygUSvNY>
- https://www.youtube.com/watch?v=G8MYGDf_9ho&t=54s
- https://www.tutorialspoint.com/pygame/pygame_mouse_events.htm
- https://labtime.ufg.br/cgames/pdf/CProgPy_Pygame.pdf
- Material da aula
- Gemini e ChatGPT